

Speech - Bullet points

SLIDE 2 - Background

- Context: we are talking about securing IT networks
- Modern networks are complex → shift from perimeter firewall to distributed firewalls
- Distributed firewalls bring benefits → total control over the network, tailored policies per area/service
- But also new challenges → higher chance of misconfiguration: blocking legitimate traffic, or worse, allowing illegitimate traffic
- In this scenario, manual configuration is risky — no guarantee of correctness

SLIDE 3 - VEREFOO

- Automated tools for FW placement and configuration, like VEREFOO, help admins manage the network without configuration errors
- VEREFOO input:
 - Network topology → described through a Service Graph (nodes and links)
 - Security Requirements → called NSRs: which traffic flows must be allowed or denied
- VEREFOO output:
 - Firewall allocation scheme + filtering policies → where firewalls are placed and how to configure them
- Main limitation: doesn't handle change — modern networks are dynamic; changing the topology or any requirement means recomputing from scratch

SLIDE 4 - React-VEREFOO

- To accommodate dynamism, React-VEREFOO was implemented: reconfigures only what actually needs to change
- Input: an existing configuration + two requirement sets — the initial (already enforced) set, and the additional requirements
- Classifies NSRs as to-be-added, to-be-removed, or kept, and reconfigures only the firewalls in the affected area
- Formal correctness still guaranteed, but much faster than the monolithic version (which always starts from scratch)
- Experimental tests: React-VEREFOO is much faster with small requirement batches, comparable to monolithic with large batches
- This last observation is the key intuition behind the next slide

SLIDE 5 - Research Question

- Use this last observation to replace VEREFOO's one-shot approach with a sequence of simple calls to React-VEREFOO, each time adding one single requirement

SLIDE 6 - Thesis Contribution: Iterative VEREFOO

- Iterative VEREFOO: introduce requirements one at a time, invoking React-VEREFOO internally at each step to compute the minimal update
- consideredProps: already-enforced requirements, passed as the initial set to the next step
- currentGraph: network configuration, updated after the most recently verified requirement

- Every iteration receives the smallest possible incremental problem
- Correctness always guaranteed — every NSR in the final set is satisfied

SLIDE 7 - Structural Comparison

- Push the incremental approach to its extreme: introduce only one NSR at a time
- Not reinventing the wheel — just using React-VEREFOO with selected inputs
- Reminder: aside from the topology, React-VEREFOO takes two batches — "old" (already enforced) and "new" requirements
- P_0 = initial requirements, $A_1, A_2, A_3 \dots$ = additional requirements introduced one by one:
 - Iteration 1: P_0 as old, A_1 as new
 - Iteration 2: $P_0 + A_1$ as old, A_2 as new
 - And so on for all N requirements
- Correctness is guaranteed by React-VEREFOO itself — we're only manipulating the interaction with it, not building a new tool

SLIDE 8 - Implementation

- Implemented inside VerefooSerializer — the shared entry point for all three variants
- No changes needed on the caller side
- Only one wrapper added: intercepts the call to the solver and manipulates the input to React-VEREFOO
- Intermediate results propagated via two structures updated at each iteration, carrying each requirement's result into the final configuration
- Everything else (MaxSMT encoding, VerefooProxy, Translator) untouched

SLIDE 9 - Validation: CESNET Topology

- Validated qualitatively on CESNET, a real academic network in the Czech Republic (23 nodes, 28 links)
- Initial configuration: 10 requirements, enforced by 3 firewalls
- 5 additional requirements introduced one at a time, tracing each iteration step by step
- Final result: 5 firewalls total configured by 4 rules, enforcing all 15 requirements
- No previously satisfied requirement was ever broken at any stage

SLIDE 10 - Validation: Iterative vs Monolithic

- Same test run with vanilla VEREFOO (monolithic) — same topology, same 15 NSRs
- Result: very similar configuration — 5 firewalls, 3 with the exact same purpose/position as the iterative solution
- Difference on the other 2 firewalls comes from visibility: iterative commits at each step, never seeing beyond the current requirement; monolithic optimizes globally
- Cost of this: one extra rule in the iterative result — negligible in any case
- Benefit: iterative never relocates an already-valid firewall — avoids operational cost and transient insecure states
- Matches how requirements accumulate in real networks — motivates using the iterative approach in dynamic environments

SLIDE 11 - Conclusions & Future Work

- The iterative approach has been validated via qualitative testing, with a comparable result to the monolithic version

- Particularly suited for highly dynamic environments where topologies or requirements change frequently
- Future work:
 - Quantitative testing to benchmark the iterative solution, over multiple configurations and topologies
 - NSR removal, not considered in this thesis
 - Optimization proposal: order NSRs based on traffic awareness